

# Lab Assignment 7: Report

## Part A: Requirement Analysis

### 1. Hotel

- **Attributes:** hotelId, name, city, rooms
- **Responsibilities:** Manages its rooms.
- **Justification:** A hotel is a physical location that contains rooms. Customers pick a hotel first.

### 2. Room

- **Attributes:** roomNumber, roomType
- **Responsibilities:** Represents a physical room in the hotel.
- **Justification:** Customers book a specific room. Multiple bookings can exist for the same room on different dates.

### 3. RoomType

- **Attributes:** typeName (Standard, Deluxe, Suite), capacity, nightlyPrice
- **Responsibilities:** Defines the pricing and capacity.
- **Justification:** Sharing capacity and price logic here avoids repeating this data for every single room.

### 4. Customer

- **Attributes:** customerId, name, email
- **Responsibilities:** Stores customer details.
- **Justification:** The system needs an entity to tie a booking to a real person.

### 5. Booking

- **Attributes:** bookingId, customer, room, checkInDate, checkOutDate, status
- **Responsibilities:** Links a customer to a room for a specific time period. Handles the reservation lifecycle.
- **Justification:** This is the core transaction. It has its own state (Pending, Confirmed, Cancelled).

## 6. Payment

- **Attributes:** paymentId, amount, status
- **Responsibilities:** Simulates processing the transaction.
- **Justification:** Keeping payment separate from booking allows clean handling of payment failures.

## 7. CancellationPolicy

- **Attributes:** fullRefundHours, partialRefundHours, partialRefundPercentage
- **Responsibilities:** Defines the rules for how much money is refunded based on the cancellation time.
- **Justification:** The PDF requires tiered rules. **Our Slabs:** 100% refund if cancelled > 72 hours before check-in, 50% refund if between 24 and 72 hours, and 0% refund if < 24 hours. Encapsulating this logic in its own class keeps the system clean.

## 8. Refund

- **Attributes:** refundId, amount, processedDate
- **Responsibilities:** Executes and stores the actual refund transaction based on the CancellationPolicy.
- **Justification:** Separates the monetary refund transaction from the booking class.

## 9. Voucher

- **Attributes:** voucherId, issueDate, bookingDetails
- **Responsibilities:** Acts as a confirmation document.
- **Justification:** Required by the problem statement as a receipt for a successful booking.

## Part B: Identify Relationships

| Relationship       | Type        | Multiplicity | Reasoning                                            |
|--------------------|-------------|--------------|------------------------------------------------------|
| Hotel and Room     | Composition | 1 to 1..*    | A hotel is made up of one or more rooms.             |
| Room and Room Type | Association | * to 1       | Many rooms can be of the same type (e.g., Standard). |

|                              |             |           |                                                                    |
|------------------------------|-------------|-----------|--------------------------------------------------------------------|
| Hotel and Booking            | Association | 1 to 0..* | A hotel can have zero to many bookings over time.                  |
| Booking and Room             | Association | * to 1    | A single room can be booked many times across different dates.     |
| Customer and Booking         | Association | 1 to 0..* | One customer can make zero or many bookings.                       |
| Booking and Voucher          | Composition | 1 to 0..1 | A confirmed booking generates exactly one voucher.                 |
| Booking and Payment          | Composition | 1 to 1    | A booking has a single payment transaction attached to it.         |
| Payment and Refund           | Association | 1 to 0..1 | A cancelled payment can have one refund.                           |
| Hotel and CancellationPolicy | Composition | 1 to 1    | The hotel defines and owns its specific cancellation policy slabs. |

## Part C: Room Availability

**Scenario:** Room 204 is booked from Sept 20 to Sept 22. Another customer wants the room from Sept 22 to Sept 24.

### Should it be allowed?

Yes, it should be allowed. In hotels, a guest checks out in the morning, and the next guest checks in during the afternoon. The dates do not conflict because the second booking starts on the exact day the first one ends.

### How to represent room availability:

A simple true/false flag (like `isAvailable`) will **not** work. A single flag only tells us if the room is occupied right now, but it cannot tell us if the room will be free next week.

Instead, a room's availability must be checked against its list of existing bookings. When a customer requests dates, the system will loop through the room's current bookings and check for overlaps.

### The overlap logic:

An overlap happens if (Requested Check-In < Existing Check-Out) **AND** (Requested Check-Out > Existing Check-In).

If there is no overlap with any existing bookings, the room is free!